

ASSIGNMENT 5

SIMPLE AND MULTI CHAT USER APPLICATION

Name: Nivetha Dhanakoti
Reg no.: 3122 22 5001 087

Aim: To be proficient in developing chat application between two users and four users using Client Server Connection.

1. Write the code using socket programming in C (K3)

Develop an application for chat using client server programming between two users using TCP.

PART A:

1. Establish the basic connection between client and the server using socket(), listen(), bind(), accept() in server side (Host A) and socket(), connect() in client side (Host B). Use send() and recv() on both sides for transferring the data.
2. Use the local machine for both host using loopback address.
3. Exchange messages between host A and host B using proper order of send() and recv() system calls.
4. Terminate the chat when any one party send "BYE" message.

PART B:

5. Extend the above program and implement chat application between any two systems in the lab.
6. Use the machines ip address for connecting the systems.
7. Use Wireshark tool, to observe the various protocols packet transfer in the network.

Server Code:

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include <unistd.h>
#include <arpa/inet.h>
```

```
#define PORT 8080
#define MAXLINE 1024
```

```
int main() {
    int server_fd, new_socket;
    struct sockaddr_in address;
    int addrlen = sizeof(address);
    char buffer[MAXLINE] = {0};
    char *hello = "Hello from server";

    // Creating socket file descriptor
    if ((server_fd = socket(AF_INET, SOCK_STREAM, 0)) == 0) {
        perror("socket failed");
        exit(EXIT_FAILURE);
    }

    // Bind the socket to the network address and port
    address.sin_family = AF_INET;
    address.sin_addr.s_addr = INADDR_ANY; // Use loopback address for local machine
    address.sin_port = htons(PORT);

    if (bind(server_fd, (struct sockaddr *)&address, sizeof(address)) < 0) {
```

```

    perror("bind failed");
    close(server_fd);
    exit(EXIT_FAILURE);
}

// Listen for incoming connections
if (listen(server_fd, 3) < 0) {
    perror("listen failed");
    close(server_fd);
    exit(EXIT_FAILURE);
}

// Accept a connection
if ((new_socket = accept(server_fd, (struct sockaddr *)&address, (socklen_t*)&addrlen)) <
0) {
    perror("accept failed");
    close(server_fd);
    exit(EXIT_FAILURE);
}

// Chat loop
while(1) {
    memset(buffer, 0, MAXLINE);
    int valread = recv(new_socket, buffer, MAXLINE, 0);
    if (valread < 0) {
        perror("recv failed");
        close(new_socket);
        close(server_fd);
        exit(EXIT_FAILURE);
    }

    printf("Client: %s\n", buffer);
    if (strcmp(buffer, "BYE") == 0) {
        printf("Client has terminated the chat.\n");
        break;
    }

    printf("Server: ");
    fgets(buffer, MAXLINE, stdin);
    buffer[strcspn(buffer, "\n")] = 0; // Remove trailing newline

    send(new_socket, buffer, strlen(buffer), 0);

    if (strcmp(buffer, "BYE") == 0) {
        printf("Server has terminated the chat.\n");
        break;
    }
}

close(new_socket);
close(server_fd);
return 0;
}

```

Client Code:

```

#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include <unistd.h>
#include <arpa/inet.h>

#define PORT 8080
#define MAXLINE 1024

int main() {
    int sock = 0;
    struct sockaddr_in serv_addr;
    char buffer[MAXLINE] = {0};

    // Creating socket file descriptor
    if ((sock = socket(AF_INET, SOCK_STREAM, 0)) < 0) {
        perror("Socket creation error");
        exit(EXIT_FAILURE);
    }

    serv_addr.sin_family = AF_INET;
    serv_addr.sin_port = htons(PORT);

    // Convert IPv4 address from text to binary form
    if (inet_pton(AF_INET, "127.0.0.1", &serv_addr.sin_addr) <= 0) {
        perror("Invalid address/ Address not supported");
        exit(EXIT_FAILURE);
    }

    // Connect to the server
    if (connect(sock, (struct sockaddr *)&serv_addr, sizeof(serv_addr)) < 0) {
        perror("Connection failed");
        exit(EXIT_FAILURE);
    }

    // Chat loop
    while(1) {
        printf("Client: ");
        fgets(buffer, MAXLINE, stdin);
        buffer[strcspn(buffer, "\n")] = 0; // Remove trailing newline

        send(sock, buffer, strlen(buffer), 0);

        if (strcmp(buffer, "BYE") == 0) {
            printf("Client has terminated the chat.\n");
            break;
        }

        memset(buffer, 0, MAXLINE);
        int valread = recv(sock, buffer, MAXLINE, 0);
        if (valread < 0) {
            perror("recv failed");
            close(sock);
            exit(EXIT_FAILURE);
        }
    }
}

```

```

        printf("Server: %s\n", buffer);
        if (strcmp(buffer, "BYE") == 0) {
            printf("Server has terminated the chat.\n");
            break;
        }
    }

    close(sock);
    return 0;
}

```

Output:

```

nivethadhanakoti@Nivethas-MacBook-Pro A5 % gcc server.c -o server
nivethadhanakoti@Nivethas-MacBook-Pro A5 % ./server
Client: Hi
Server: Hello SSN
Client: Nice meeting you
Server: Same here
Client: BYE
Client has terminated the chat.

nivethadhanakoti@Nivethas-MacBook-Pro A5 % gcc client.c -o client
nivethadhanakoti@Nivethas-MacBook-Pro A5 % ./client
Client: Hi
Server: Hello SSN
Client: Nice meeting you
Server: Same here
Client: BYE
Client has terminated the chat.

```

2. Write the code using socket programming in C (K3)

Develop an application for chat using client server programming between four users using TCP.

PART A:

1. Establish the basic connection between client and the server using socket(), listen(), bind(), accept() in server side (Host A) and socket(), connect() in client side (Host B). Use send() and recv() on both sides for transferring the data.
2. Use fork() or FD_SET(), Select() system calls for multi user.
3. Use the local machine for both host using loopback address.
4. Exchange messages between host A and host B using proper order of send() and recv() system calls.
5. Terminate the chat when any one party send "BYE" message.

PART B:

6. Extend the above program and implement chat application between any four systems in the lab.
7. Use the machines ip address for connecting the systems.
8. Use Wireshark tool, to observe the various protocols packet transfer in the network.

Server Code:

```

#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include <unistd.h>
#include <arpa/inet.h>
#include <sys/socket.h>
#include <sys/select.h>
#include <errno.h>
#define PORT 8080
#define MAXLINE 1024

```

```

#define MAX_CLIENTS 4

int main() {
    int server_fd, new_socket, client_socket[MAX_CLIENTS], max_clients =
MAX_CLIENTS, activity, i, valread, sd;
    int max_sd;
    struct sockaddr_in address;
    char buffer[MAXLINE];
    fd_set readfds;

    for (i = 0; i < max_clients; i++)
        client_socket[i] = 0;

    if ((server_fd = socket(AF_INET, SOCK_STREAM, 0)) == 0)
    {
        perror("socket failed");
        exit(EXIT_FAILURE);
    }

    address.sin_family = AF_INET;
    address.sin_addr.s_addr = INADDR_ANY;
    address.sin_port = htons(PORT);

    if (bind(server_fd, (struct sockaddr *)&address, sizeof(address)) < 0)
    {
        perror("bind failed");
        close(server_fd);
        exit(EXIT_FAILURE);
    }

    if (listen(server_fd, 3) < 0)
    {
        perror("listen failed");
        close(server_fd);
        exit(EXIT_FAILURE);
    }

    printf("Waiting for connections...\n");

    while(1)
    {
        FD_ZERO(&readfds);
        FD_SET(server_fd, &readfds);
        max_sd = server_fd;
        for (i = 0; i < max_clients; i++)
        {
            sd = client_socket[i];
            if (sd > 0)
                FD_SET(sd, &readfds);
            if (sd > max_sd)
                max_sd = sd;
        }

        activity = select(max_sd + 1, &readfds, NULL, NULL, NULL);
    }
}

```

```

if ((activity < 0) && (errno != EINTR))
    printf("select error");

if (FD_ISSET(server_fd, &readfds))
{
    if ((new_socket = accept(server_fd, (struct sockaddr *)&address,
(socklen_t*)&address)) < 0)
    {
        perror("accept");
        exit(EXIT_FAILURE);
    }

    printf("New connection, socket fd is %d, ip is : %s, port : %d \n",new_socket,
inet_ntoa(address.sin_addr), ntohs(address.sin_port));

    for (i = 0; i < max_clients; i++)
    {
        if (client_socket[i] == 0)
        {
            client_socket[i] = new_socket;
            printf("Adding to list of sockets as %d\n", i);
            break;
        }
    }
}

for (i = 0; i < max_clients; i++)
{
    sd = client_socket[i];

    if (FD_ISSET(sd, &readfds))
    {
        if ((valread = read(sd, buffer, MAXLINE)) == 0)
        {
            getpeername(sd, (struct sockaddr*)&address, (socklen_t*)&address);
            printf("Host disconnected, ip %s, port %d \n",
                inet_ntoa(address.sin_addr), ntohs(address.sin_port));
            close(sd);
            client_socket[i] = 0;
        }
        else
        {
            buffer[valread] = '\0';
            printf("Client %d: %s\n", i + 1, buffer);
            for (int j = 0; j < max_clients; j++)
            {
                if (client_socket[j] != 0 && j != i)
                    send(client_socket[j], buffer, strlen(buffer), 0);
            }

            if (strcmp(buffer, "BYE") == 0)
            {
                printf("Chat terminated by client %d\n", i + 1);
                close(sd);
                client_socket[i] = 0;
            }
        }
    }
}

```

```

    }
    }
    }
    }
    return 0;
}

```

Client Code:

```

#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include <unistd.h>
#include <arpa/inet.h>
#define PORT 8080
#define MAXLINE 1024

int main() {
    int sock = 0;
    struct sockaddr_in serv_addr;
    char buffer[MAXLINE] = {0};

    if ((sock = socket(AF_INET, SOCK_STREAM, 0)) < 0)
    {
        perror("Socket creation error");
        exit(EXIT_FAILURE);
    }

    serv_addr.sin_family = AF_INET;
    serv_addr.sin_port = htons(PORT);

    if (inet_pton(AF_INET, "127.0.0.1", &serv_addr.sin_addr) <= 0)
    {
        perror("Invalid address/ Address not supported");
        exit(EXIT_FAILURE);
    }

    if (connect(sock, (struct sockaddr *)&serv_addr, sizeof(serv_addr)) < 0)
    {
        perror("Connection failed");
        exit(EXIT_FAILURE);
    }

    while(1)
    {
        printf("Client: ");
        fgets(buffer, MAXLINE, stdin);
        buffer[strcspn(buffer, "\n")] = 0;
        send(sock, buffer, strlen(buffer), 0);

        if (strcmp(buffer, "BYE") == 0)
        {
            printf("Client has terminated the chat.\n");
            break;
        }
    }
}

```

```

memset(buffer, 0, MAXLINE);
int valread = recv(sock, buffer, MAXLINE, 0);
if (valread < 0)
{
    perror("recv failed");
    close(sock);
    exit(EXIT_FAILURE);
}

printf("Server: %s\n", buffer);
if (strcmp(buffer, "BYE") == 0)
{
    printf("Server has terminated the chat.\n");
    break;
}
}

close(sock);
return 0;
}

```

Output:

```

nivethadhanakoti@Nivethas-MacBook-Pro A5 % gcc multiserver.c -o multis
nivethadhanakoti@Nivethas-MacBook-Pro A5 % ./multis
Waiting for connections...
New connection, socket fd is 4, ip is : 127.0.0.1, port : 0
Adding to list of sockets as 0
New connection, socket fd is 5, ip is : 127.0.0.1, port : 0
Adding to list of sockets as 1
New connection, socket fd is 6, ip is : 127.0.0.1, port : 0
Adding to list of sockets as 2
Client 1: Hi
Client 2: Hello
Client 3: Good Morning
Client 1: Nice meeting you
Client 2: bye
Client 2: BYE
Chat terminated by client 2
Client 3: BYE
Chat terminated by client 3
Client 1: BYE
Chat terminated by client 1
□
nivethadhanakoti@Nivethas-MacBook-Pro A5 % gcc multiclient.c -o multic
nivethadhanakoti@Nivethas-MacBook-Pro A5 % ./multic
Client: Hi
Server: Hello
Client: Nice meeting you
Server: Good Morning
Client:
Server: byeBYEBYE
Client: BYE
Client has terminated the chat.

```

```
nivethadhanakoti@Nivethas-MacBook-Pro A5 % gcc multicient.c -o multict2
nivethadhanakoti@Nivethas-MacBook-Pro A5 % ./multict2
Client: Hello
Server: Hi
Client: bye
Server: Good MorningNice meeting you
Client: BYE
Client has terminated the chat.

nivethadhanakoti@Nivethas-MacBook-Pro A5 % gcc multicient.c -o multict3
nivethadhanakoti@Nivethas-MacBook-Pro A5 % ./multict3
Client: Good Morning
Server: HiHello
Client:
Server: Nice meeting youbyeBYE
Client: BYE
Client has terminated the chat.
```

Learning Outcomes:

I have learnt to be proficient in developing chat application between two users and four users using Client Server Connection.